

SCHEMA DI RIPASSO COMPLETO — ITS Mobile App Development

MATERIA 1 — Sviluppo Web Client-Side & HTTP/REST

HTML5 — Form e Input

Tag **<form>**:

- **action** — URL o file a cui inviare i dati (es. script PHP o endpoint API)
- **method GET** — dati visibili nell'URL, spazio limitato, meno sicuro. Adatto per ricerche
- **method POST** — dati nel body della richiesta, più sicuro. Adatto per login e dati sensibili
- **target** — dove aprire la risposta (**_blank** = nuova scheda)

Tipi di **<input>**:

- **text** — campo di testo normale
- **password** — testo mascherato da pallini
- **radio** — scelta singola tra più opzioni (devono avere lo stesso attributo **name**)
- **checkbox** — scelte multiple indipendenti (attributo **checked** per il valore iniziale)
- **file** — caricamento file dalla memoria locale
- **submit** — bottone che avvia l'invio del form
- **reset** — azzera tutti i campi ai valori iniziali

Altri elementi:

- **<label for="ID">** — etichetta legata all'input tramite ID; fondamentale per l'accessibilità
- **<textarea rows cols>** — area di testo multilinea ed espandibile
- **<select> + <option>** — menu a tendina; attributo **multiple** per selezione multipla
- **** — lista ordinata (numeri/lettere) | **** — lista non ordinata (bullet) | **** — elemento lista

CSS — Specificità e Layout

Responsive Design — approccio che permette all'interfaccia di adattarsi automaticamente a dispositivi e schermi diversi (PC, tablet, smartphone), modificando dinamicamente layout e disposizione degli elementi.

Calcolo Specificità (Inline, ID, Classi, Tag):

- Inline style `style="..."` → (1, 0, 0, 0) — peso massimo
- ID `#header` → (0, 1, 0, 0)
- Classi / attributi / pseudo-classi `.btn:hover [type="text"]` → (0, 0, 1, 0)
- Tag / pseudo-elementi `div p ::before` → (0, 0, 0, 1)
- Ereditarietà → specificità zero
- `!important` — scavalca qualsiasi punteggio, usare con parsimonia

📌 **ESAME** — Selettore `ul#players li.winner`: 1 ID + 1 classe + 2 tag = (0, 1, 1, 2)

Proprietà CSS base:

- `background-color` — modifica lo sfondo di un elemento
 - `color` — modifica il colore del testo
 - `padding` — spazio interno tra il contenuto e il bordo dell'elemento
-

Formati Immagine

Raster (pixel):

- **JPEG** — lossy (con perdita), ideale per foto, NON supporta trasparenza
- **PNG** — lossless (senza perdita), supporta trasparenza, ideale per loghi e icone
- **GIF** — max 256 colori, animazioni semplici, trasparenza base
- **WebP** — formato moderno, alta compressione, supporta trasparenza + animazioni

Vettoriale:

- **SVG** — scalabile infinitamente senza perdere qualità, basato su XML. Standard per export da Figma verso Android e iOS. Leggerissimo

Web server locale — per testare su dispositivi diversi serve un software web server.

Entrambi i dispositivi devono essere sulla stessa rete Wi-Fi e bisogna conoscere l'IP locale della macchina server.

JavaScript — Fondamenti

Variabili:

- `let` — ambito di blocco, il valore può essere modificato

- `const` — valore fisso e immutabile dopo la creazione
- Nomi: iniziano con lettere, `_` o `$`, case-sensitive, formato camelCase

Tipi primitivi: `string` | `number` | `boolean` | `undefined` | `null`

Coercizione — se sommi stringa e numero (`"5" + 10`), JS converte il numero in stringa e concatena → risultato: `"510"`

Funzioni:

- Standard: `function somma(a, b) { return a + b; }`
- Arrow function: `const prodotto = (a, b) => a * b;`
- `return` restituisce un valore usabile | `console.log()` mostra solo nel terminale

Array — metodi principali:

- `push()` aggiunge in fondo | `pop()` toglie dal fondo
- `unshift()` aggiunge in testa | `shift()` toglie dalla testa
- `includes(valore)` → true/false | `indexOf(valore)` → indice o -1
- `map()` — crea nuovo array modificando ogni elemento
- `filter()` — crea nuovo array filtrato per condizione
- `slice(inizio, fine)` — ritaglia parte dell'array senza modificarlo

Oggetti:

- Accesso: `oggetto.proprieta` oppure `oggetto['proprieta']`
- Modifica/aggiunta: `oggetto.chiave = valore` | Eliminazione: `delete oggetto.chiave`
- `this` — dentro un metodo, riferisce l'oggetto stesso
- `Object.keys(obj)` — array di tutte le chiavi
- `Object.freeze(obj)` — rende l'oggetto completamente immutabile

Timing:

- `setTimeout(fn, ms)` — esegue una volta dopo il delay. Si cancella con `clearTimeout(id)`
- `setInterval(fn, ms)` — esegue ciclicamente ogni tot ms. Si ferma con `clearInterval(id)`

JSON:

- `JSON.stringify(obj)` — converte oggetto JS in stringa JSON
- `JSON.parse(string)` — converte stringa JSON in oggetto JS
- Le chiavi vanno tra doppi apici. Non supporta funzioni, commenti o oggetti `Date`

Fetch API (richieste asincrone):

- `fetch(url)`
- `.then(res => res.json())`
- `.then(data => console.log(data))`

```
.catch(err => console.error(err));
```

Architettura HTTP & REST API

Definizione — un'API fa da ponte tra due software. Nello standard REST, la comunicazione è stateless Client-Server su protocollo HTTP.

Verbi HTTP:

- **GET** — recupera dati (no body, parametri nell'URL)
- **POST** — crea una nuova risorsa (dati nel body)
- **PUT** — aggiorna/sovrascrive interamente una risorsa esistente
- **DELETE** — cancella una risorsa dal server

Struttura della comunicazione:

- **Request (Client → Server):** Metodo + Header (es. `Content-Type: application/json`) + Body (JSON, non presente nelle GET)
- **Response (Server → Client):** Status Code + Headers + Content (JSON)

Status Code:

- `2xx` (200 OK, 201 Created) — successo
- `3xx` — reindirizzamento
- `4xx` (404 Not Found, 400 Bad Request) — errore del Client
- `5xx` (500 Internal Server Error) — errore del Server

Postman — tool per testare REST API senza scrivere codice. Simula richieste, verifica status code, body e tempi. Organizza le richieste in "Collection".

MATERIA 2 — Python, OOP & Database SQL

Programmazione Orientata agli Oggetti (OOP)

- **Classe** — lo "stampo": definisce proprietà e metodi comuni
- **Oggetto / Istanza** — la copia concreta creata dalla classe, con i propri valori specifici
- `__init__` — costruttore: eseguito automaticamente alla creazione dell'oggetto, inizializza le proprietà tramite `self`
- `self` — puntatore all'istanza corrente
- **Ereditarietà** — classe figlia eredita metodi e attributi dalla classe madre senza riscrivere codice

- **super().__init__()** — richiama il costruttore della classe padre dalla classe figlia

Esempio:

```

• class Veicolo:
•     def __init__(self, marca):
•         self.marca = marca
•
•     def muovi(self):
•         print(f"Il veicolo {self.marca} si muove!")
•
• class Auto(Veicolo):
•     def __init__(self, marca, modello):
•         super().__init__(marca)
•         self.modello = modello
•
• auto1 = Auto("Fiat", "Panda")
auto1.muovi() # Output: Il veicolo Fiat si muove!

```

Tipi di JOIN in SQL

- **INNER JOIN** — restituisce SOLO le righe con match in ENTRAMBE le tabelle
- **LEFT JOIN** — tutte le righe della tabella sinistra + corrispondenze destra (NULL se assente)
- **RIGHT JOIN** — tutte le righe della tabella destra + corrispondenze sinistra (NULL se assente)

 **ESAME — INNER JOIN = solo match in entrambe. Risposta più comune nei quiz.**

SQLite3 in Python

SQLite3 — libreria nativa Python per database relazionali leggeri su file locale, zero configurazione.

int() — converte stringa in intero. Esempio: **int("10") → 10**

Flusso operativo:

- import sqlite3
- conn = sqlite3.connect("mio_database.db") # crea il file se non esiste
- cur = conn.cursor() # cursore per eseguire SQL
-
- cur.execute("""
- CREATE TABLE IF NOT EXISTS utenti (
- id INTEGER PRIMARY KEY AUTOINCREMENT,

- nome TEXT,
- email TEXT
-)
- """)
- conn.commit()
-
- cur.execute("INSERT INTO utenti (nome, email) VALUES (?, ?)", ("Mario", "mario@email.com"))
- conn.commit() # obbligatorio dopo ogni modifica
-
- cur.execute("SELECT * FROM utenti")
- for utente in cur.fetchall():
- print("ID:", utente[0], "| Nome:", utente[1])
-

conn.close()

Riepilogo passi:

1. `sqlite3.connect('db.db')` — connessione/creazione
2. `conn.cursor()` — cursore
3. `cur.execute("SQL")` — esecuzione comando
4. `conn.commit()` — salva modifiche ← obbligatorio dopo scritture
5. `cur.fetchall()` — lista di tuple con i risultati
6. `conn.close()` — chiude in sicurezza

MATERIA 3 — Controllo Versione (Git & GitHub)

Configurazione e Flusso Base

- `git config user.name "Nome"`

`git config user.email "mail@email.com"`

Comandi principali:

- `git init` — inizializza repository Git nella cartella corrente
- `git clone <url>` — copia un repo remoto in locale
- `git add .` — porta tutte le modifiche in staging area
- `git add <file>` — porta un file specifico in staging
- `git add -p` — aggiunge modifiche in modo interattivo, riga per riga
- `git commit -m "messaggio"` — salva snapshot nella cronologia locale
- `git push origin main` — invia i commit al server remoto
- `git log` — mostra la cronologia di tutti i commit (ID, autore, data, messaggio)

- `git blame <file>` — mostra chi ha modificato ogni riga del file e con quale commit
-

Gestione Errori e Reset

- `git commit --amend` — modifica l'ultimo commit (messaggio o file dimenticati). Con `--reset-author` aggiorna anche l'autore
 - `git reset <id>` — annulla commit successivi ma MANTIENE le modifiche ai file locali
 - `git reset --hard <id>` — ⚠ DISTRUTTIVO: cancella commit E tutte le modifiche ai file per sempre. Riporta il progetto allo stato esatto del commit specificato
-

Branch e Collaborazione

- `git checkout -b funzionalita-x` — crea nuovo branch E si sposta su di esso immediatamente
- `git checkout main` — torna al branch principale
- `git remote add nome url_ssh` — collega il repo locale a un server remoto
- `git remote -v` — mostra tutti i remote configurati
- `git fetch <remote>` — scarica dati dal remote SENZA fare merge
- `git pull <remote> <branch>` — scarica + merge immediato nel branch locale (= fetch + merge)
- `git push <remote> <branch>` — invia commit locali al remote
- `git merge` — unisce due branch mantenendo la cronologia completa in ordine cronologico
- `git rebase` — unisce branch riscrivendo la storia in modo lineare (i propri commit sopra al branch di destinazione)
- `git cherry-pick <id>` — preleva un singolo commit specifico da un altro branch e lo applica sul branch corrente
- **Fork** — copia indipendente del repo altrui sul proprio profilo GitHub, per lavorare liberamente
- **Pull Request (PR)** — richiesta formale su GitHub per proporre le proprie modifiche al progetto originale

📌 **ESAME** — `git checkout -b nome` crea E sposta. `git branch nome` crea solo, senza spostarsi.

MATERIA 4 — Cybersecurity, Sistemi & Normative

Fondamenti — Triade CIA e Rischio

📌 **Formula del Rischio: $R = P \times D$ (Probabilità $\times$ Danno)**

Triade CIA:

- **Confidenzialità** — dati accessibili solo agli utenti autorizzati (es. crittografia)
- **Integrità** — dati non alterati né corrotti, né da attacchi né da errori accidentali
- **Disponibilità** — sistemi e dati sempre accessibili e fruibili rapidamente dagli aventi diritto

Profili degli attaccanti:

- **Script Kiddies** — dilettanti che usano tool altrui senza capire il funzionamento tecnico
 - **White Hat** — hacker etici che testano sistemi con permesso esplicito per migliorare le difese
 - **Gray Hat** — violano senza permesso ma senza scopi distruttivi; spesso segnalano le falle in cambio di compenso o notorietà
-

Crittografia Simmetrica — DES e AES

DES (1976) — OBSOLETO:

- Blocchi da 64 bit, chiave da 64 bit (ma solo 56 bit effettivi, 8 bit di parità)
- Struttura a rete di Feistel, 16 round
- Violato in 22 ore nel 1999 dal sistema Deep Crack — spazio chiavi 2^{56} troppo piccolo
- Triple DES (3DES) — ripete il processo 3 volte, sicuro ma troppo lento

AES (2001) — STANDARD MODERNO (HTTPS, WPA2/WPA3, VPN):

- Blocchi fissi da 128 bit, organizzati in una matrice 4×4 di byte chiamata "State"
- Chiave da 128 bit $\rightarrow$ 10 round
- Chiave da 192 bit $\rightarrow$ 12 round
- Chiave da 256 bit $\rightarrow$ 14 round
- La chiave principale viene espansa tramite Key Schedule in tante sottochiavi quanti sono i round + 1

Fasi dei round AES (in ordine):

1. **SubBytes** — sostituzione non lineare di ogni byte tramite tabella S-Box (16×16)
2. **ShiftRows** — rotazione righe verso sinistra: riga 0 ferma, riga 1 = 1 posizione, riga 2 = 2 posizioni, riga 3 = 3 posizioni
3. **MixColumns** — moltiplicazione colonne in $GF(2^8)$ per diffondere i bit $\rightarrow$ **ASSENTE nel round FINALE**
4. **AddRoundKey** — XOR tra lo State e la sottochiave del round corrente

Round iniziale: solo AddRoundKey **Round intermedi:** SubBytes → ShiftRows → MixColumns → AddRoundKey **Round finale:** SubBytes → ShiftRows → AddRoundKey (senza MixColumns)

📌 **ESAME — MixColumns ASSENTE nell'ultimo round. Dettaglio sempre testato.**

Macchina Enigma

- Dispositivo elettromeccanico tedesco a rotori per cifrare messaggi
 - Componenti: **Plugboard** (scambio coppie di lettere) + **3 Rotori** (sostituzioni) + **Riflettore** (rimanda il segnale indietro)
 - Ad ogni tasto premuto il primo rotore avanza di una posizione, cambiando la mappatura
 - Una lettera non può mai essere cifrata in se stessa (per via del riflettore)
 - Violata nel 1932 da Marian Rejewski (Polonia), poi a Bletchley Park da Alan Turing e Gordon Welchman
-

Password, Permessi Linux, Rete e GDPR

Tecniche di attacco alle password:

- **Default Password** — credenziali di fabbrica mai cambiate, facilmente reperibili online
- **Social Engineering / Password Profiling** — studio della vittima per generare combinazioni probabili
- **Attacco a dizionario** — lista massiva di password reali già violate in precedenza
- **Brute Force** — tutte le combinazioni possibili, sistematicamente. Ha successo matematico garantito nel tempo
- **Credential Stuffing** — credenziali rubate da un sito testate su altri siti (gli utenti riutilizzano le stesse password)

Entropia: $K = \Sigma^L$ (simboli possibili elevati alla lunghezza). Meno ripetizioni = più entropia = più sicurezza.

- < 28 bit = molto debole | 28–35 = debole | 36–59 = ragionevole | 60–127 = **forte** | 128+ = overkill

Permessi Linux (chmod) — valori fissi:

- r (read) = 4 | w (write) = 2 | x (execute) = 1
- Classi: **u** (User/proprietario) | **g** (Group) | **o** (Others)
- Calcolo: somma dei valori per ogni classe
- 7 = rwx | 6 = rw- | 5 = r-x | 4 = r--
- **chmod** — cambia i permessi | **chown** — cambia il proprietario

📌 **ESAME — chmod 764: User = rwx (7) | Group = rw- (6) | Others = r-- (4)**

Networking:

- **Gateway predefinito** — router che instrada il traffico verso reti esterne (Internet) quando l'IP di destinazione è fuori dalla LAN

Certificati digitali X.509:

- **PKI (Public Key Infrastructure)** — architettura di sicurezza per gestire certificati
- **Certificate Authority (CA)** — entità terza fidata che emette e firma digitalmente i certificati X.509. Verifica l'identità del richiedente prima dell'emissione, assumendosi la responsabilità pubblica dell'associazione tra chiave pubblica e identità reale

GDPR — Dati Particolari (Art. 9): Includono: origine razziale o etnica, opinioni politiche, convinzioni religiose o filosofiche, appartenenza sindacale, dati genetici, dati biometrici, dati sanitari, orientamento sessuale.

- **Regola base:** VIETATO trattarli
- **Eccezioni:** consenso esplicito scritto dell'interessato, oppure interesse pubblico rilevante regolato dalla legge (es. sanità pubblica)
- **Minimizzazione** — raccogliere solo i dati strettamente necessari
- **Data Retention** — conservare solo per il tempo minimo necessario
- **Accountability** — documentare e dimostrare costantemente la conformità
- **Privacy by Design & by Default** — protezione integrata fin dalla progettazione, massime tutele attive di default

GDPR — Art. 32 (Sicurezza del Trattamento): obbliga il titolare ad adottare misure tecniche adeguate: pseudonimizzazione e cifratura, garanzia di riservatezza/integrità/disponibilità, backup e ripristino, test periodici, controllo accessi con autenticazione forte.

Marchi — protezione legale per 10 anni, rinnovabile. **Brevetti** — protezione per 20 anni dalla data di deposito. Requisiti: Novità + Attività Inventiva + Applicabilità Industriale.

MATERIA 5 — Architettura Software Mobile

I 3 Layer dell'App Mobile

- **Presentation Layer (UI)** — mostra le schermate e cattura gli input dell'utente → *il cameriere che porta i piatti*
- **Domain Layer (Logica di Business)** — contiene le regole di business e i casi d'uso. È totalmente isolato da UI e fonti dati → *lo chef che cucina*
- **Data Layer (Dati)** — gestisce chiamate HTTP e database locali, nasconde i dettagli tecnici agli altri layer → *la dispensa/fornitore*

Stato dell'applicazione — insieme dei dati dinamici che cambiano nel tempo e modificano la UI (input utente, dati scaricati da API). Va separato dall'interfaccia. Principio: **Single Source of Truth**.

Modularizzazione — scomposizione del codice in moduli indipendenti per evitare dipendenze circolari e massimizzare la manutenibilità.

Struttura cartelle consigliata:

- **features/** — sottocartelle per funzionalità (auth/, feed/, profile/), ognuna con presentation/, domain/, data/
 - **shared/** — widget e utility riutilizzabili
 - **core/** — elementi globali (router, temi, traduzioni)
-

Principi SOLID

- **S — Single Responsibility** — una classe deve avere una sola responsabilità (evitare classi "tuttofare")
 - **O — Open/Closed** — aperta all'estensione (si crea una sottoclasse), chiusa alla modifica (non si tocca il codice vecchio)
 - **L — Liskov Substitution** — le sottoclassi devono poter sostituire le classi base senza rompere il programma
 - **I — Interface Segregation** — meglio molte interfacce piccole e specifiche che una sola gigante e generica
 - **D — Dependency Inversion** — le classi di alto livello non dipendono da quelle di basso livello; entrambe dipendono da astrazioni (interfacce)
-

State Management — Redux, BLoC, GetIt, Singleton

Redux / AsyncRedux:

- Pattern che centralizza l'intero stato dell'app in un unico contenitore immutabile chiamato **Store**
- Lo stato non si modifica direttamente: bisogna inviare delle **Actions**
- **AsyncRedux** — variante pensata per Flutter che gestisce nativamente le operazioni asincrone (chiamate API) riducendo drasticamente il codice ripetitivo (boilerplate) rispetto al Redux classico

BLoC (Business Logic Component):

- Pattern basato su flussi di dati reattivi (Streams)
- La UI non conosce la logica: si limita a inviare **Eventi** (es. ClickBottone) al BLoC
- Il BLoC intercetta l'evento, esegue la logica e restituisce un nuovo **Stato** (es. StatoCaricamento → StatoSuccesso)
- La UI, restando in ascolto, si ridisegna automaticamente al variare dello stato

GetIt — Dependency Injection:

- La **Dependency Injection** è un pattern che evita che una classe crei da sola le istanze dei servizi da cui dipende, riducendo l'accoppiamento
- **GetIt** è un service locator leggero per Flutter. All'avvio dell'app, tutti i servizi globali (client API, database) vengono registrati in un contenitore centrale. Qualsiasi widget può poi richiedere l'istanza condivisa ovunque, senza ricrearla né passarla manualmente nell'albero dei componenti
- Tipi di registrazione: `registerSingleton` (una sola istanza per sempre) | `registerFactory` (nuova istanza ogni volta) | `registerLazySingleton` (creata solo al primo utilizzo)

Singleton Pattern:

- Garantisce che di una classe esista una **sola istanza** nell'intera app
 - Implementazione Swift: `static let shared = NomeClasse() + private init()`
 - Implementazione Dart/Flutter: costruttore factory con istanza statica privata
 - Usato per: NetworkManager, database locali, configurazioni globali
-

MATERIA 6 — Flutter & Dart

Fondamenti di Flutter e Dart

Flutter — framework open-source Google per app native multi-piattaforma (Android, iOS, Web, Desktop) da un unico codice sorgente. Converte il codice direttamente in linguaggio macchina nativo, senza bridge intermedi. Include motore grafico autonomo, widget pronti e strumenti di debug.

Dart — linguaggio fortemente tipizzato (simile a TypeScript) su cui si basa Flutter. Punto di partenza obbligatorio: `void main()`

Variabili in Dart:

- `const` — costante a tempo di compilazione (valore noto prima dell'esecuzione)
- `final` — costante a tempo di esecuzione (assegnata una sola volta dopo l'avvio, es. un valore letto da internet)
- `late` — variabile non nulla con inizializzazione posticipata (promessa che il valore sarà assegnato prima dell'uso)
- `?` (nullable) — variabile che può contenere anche `null` (es. `String? nome`)
- `dynamic` — disattiva il controllo rigido dei tipi (da evitare, preferire `var` con inferenza)

Widget — mattone base fondamentale di Flutter. Qualsiasi elemento grafico visibile o logico (layout, bottoni, testi, allineamenti) è un Widget riutilizzabile.

Storage Locale in Flutter

Questa è un'area importante per l'esame. Esistono due approcci principali.

SharedPreferences (**shared_preferences**):

- Salva piccole quantità di dati persistenti sul dispositivo in formato chiave-valore
- I dati sono salvati **in chiaro**, non cifrati
- Adatto per: preferenze utente, impostazioni dell'app, flag (es. "utente ha già visto il tutorial")
- Non adatto per: password, token di autenticazione, dati sensibili
- Funziona su: Android (SharedPreferences), iOS (NSUserDefaults)

dart

- // Scrittura
- final prefs = await SharedPreferences.getInstance();
- await prefs.setString('username', 'Mario');
- await prefs.setInt('counter', 5);
- await prefs.setBool('dark_mode', true);
-
- // Lettura
- final username = prefs.getString('username') ?? 'Ospite';
- final counter = prefs.getInt('counter') ?? 0;
-
- // Eliminazione
- await prefs.remove('username');

await prefs.clear(); // cancella tutto

flutter_secure_storage:

- Salva dati sensibili in modo **completamente cifrato e automatico**
- Su iOS usa il **KeyChain** | Su Android usa il **KeyStore**
- Adatto per: token JWT, password, chiavi API, dati di autenticazione
- I dati sopravvivono alla disinstallazione dell'app su iOS (KeyChain persistente)

dart

- const storage = FlutterSecureStorage();
-
- // Scrittura
- await storage.write(key: 'auth_token', value: 'abc123xyz');
-
- // Lettura
- final token = await storage.read(key: 'auth_token');
-
- // Eliminazione

- `await storage.delete(key: 'auth_token');`
`await storage.deleteAll();` // cancella tutto

📌 **Differenza chiave: SharedPreferences = dati in chiaro, piccole preferenze.**
flutter_secure_storage = dati cifrati, informazioni sensibili.

Navigazione in Flutter

go_router — pacchetto moderno per la navigazione:

- Si definisce una lista strutturata di route con path
- Il path `/` è la rotta iniziale (home)
- La proprietà `redirect` permette di bloccare o reindirizzare (es. se l'utente non è loggato, rimanda al login)
- Supporta navigazione dichiarativa e deep linking

dart

- `final router = GoRouter(`
- `routes: [`
- `GoRoute(path: '/', builder: (context, state) => HomeScreen()),`
- `GoRoute(path: '/login', builder: (context, state) => LoginScreen()),`
- `GoRoute(path: '/profile/:id', builder: (context, state) => ProfileScreen()),`
- `],`
- `redirect: (context, state) {`
- `final isLoggedIn = AuthService.isLoggedIn;`
- `if (!isLoggedIn && state.location != '/login') return '/login';`
- `return null;`
- `},`

`);`

Dependency Injection con GetIt in Flutter

Il problema senza DI: ogni widget crea da solo i propri servizi → duplicazione, accoppiamento, impossibile testare.

La soluzione con GetIt:

dart

- `// 1. Registrazione all'avvio dell'app (main.dart o setup file)`
- `final getIt = GetIt.instance;`
-
- `void setupDependencies() {`
- `// Singleton: una sola istanza per tutta la vita dell'app`
- `getIt.registerSingleton<ApiService>(ApiService());`

-
- // LazySingleton: creato solo al primo utilizzo
- getIt.registerLazySingleton<DatabaseService>(() => DatabaseService());
-
- // Factory: nuova istanza ogni volta che viene richiesta
- getIt.registerFactory<UserRepository>(() => UserRepository(getIt<ApiService>()));
- }
-
- // 2. Utilizzo in qualsiasi widget o classe
- class MyWidget extends StatelessWidget {
- final apiService = getIt<ApiService>(); // recupera l'istanza registrata
-
- @override
- Widget build(BuildContext context) { ... }

}

Vantaggi: nessun passaggio manuale di dipendenze nell'albero dei widget, codice più pulito, facile da testare sostituendo le implementazioni reali con mock.

AsyncRedux in Flutter

Redux classico — tutto lo stato dell'app in un unico Store immutabile. Le Actions descrivono cosa è successo. I Reducer calcolano il nuovo stato.

AsyncRedux — versione semplificata di Redux per Flutter che gestisce le operazioni asincrone (chiamate API, database) senza la complessità del middleware del Redux classico.

Componenti principali:

- **Store** — contenitore unico e immutabile dello stato globale dell'app
- **AppState** — classe che rappresenta l'intero stato dell'applicazione
- **Action** — classe che descrive un'intenzione di modifica. Può essere sincrona o asincrona
- **Reducer** — funzione pura che riceve lo stato attuale e un'Action e restituisce il nuovo stato
- **StoreConnector** — widget che si connette allo Store e si ridisegna quando lo stato cambia

dart

- // Definizione dello stato
- class AppState {
- final List<User> users;
- final bool isLoading;
- AppState({required this.users, required this.isLoading});
- }

-
- // Action asincrona (la magia di AsyncRedux)
- class FetchUsersAction extends ReduxAction<AppState> {
- @override
- Future<AppState> reduce() async {
- // Puoi fare chiamate async direttamente nell'Action
- final users = await ApiService().getUsers();
- return state.copyWith(users: users, isLoading: false);
- }
-
- @override
- void before() => dispatch(SetLoadingAction(true)); // prima del fetch
- }
-
- // Nel widget
- StoreConnector<AppState, List<User>>(
- converter: (store) => store.state.users,
- builder: (context, users) => ListView(...),

)

BLoC Pattern in Flutter

BLoC (Business Logic Component) — separa nettamente la UI dalla logica di business usando flussi di dati reattivi (Streams).

Flusso: UI invia **Evento** → BLoC riceve l'evento → esegue la logica → emette nuovo **Stato** → UI si ridisegna

Componenti:

- **Event** — classe che rappresenta un'azione dell'utente (es. `LoadUsersEvent`, `LoginButtonPressed`)
- **State** — classe che rappresenta la situazione attuale (es. `UsersLoading`, `UsersLoaded`, `UsersError`)
- **BLoC** — classe che mappa eventi in stati

dart

- // Events
- abstract class UserEvent {}
- class LoadUsers extends UserEvent {}
-
- // States
- abstract class UserState {}
- class UserLoading extends UserState {}
- class UserLoaded extends UserState {
- final List<User> users;

- UserLoaded(this.users);
- }
- class UserError extends UserState {}
-
- // BLoC
- class UserBloc extends Bloc<UserEvent, UserState> {
- UserBloc() : super(UserLoading()) {
- on<LoadUsers>((event, emit) async {
- emit(UserLoading());
- try {
- final users = await ApiService().getUsers();
- emit(UserLoaded(users));
- } catch (e) {
- emit(UserError());
- }
- });
- }
- }
-
- // Nel widget
- BlocBuilder<UserBloc, UserState>(
- builder: (context, state) {
- if (state is UserLoading) return CircularProgressIndicator();
- if (state is UserLoaded) return ListView(...);
- return Text('Errore');
- },

)

"Come può essere Dio, team data" — Pattern Architetture dei Dati

Questa sezione riguarda come gestire i dati in modo centralizzato e affidabile nell'app.

Repository Pattern:

- Strato intermedio tra il Domain Layer e il Data Layer
- La logica di business chiede i dati al Repository, senza sapere se vengono da un'API remota, da un database locale o da una cache
- Rende il codice testabile (si può sostituire il repository reale con uno mock)

Data Source:

- Sorgente concreta dei dati: RemoteDataSource (chiamate HTTP) o LocalDataSource (database locale)
- Il Repository orchestra le due sorgenti (es. "prima guarda nella cache locale, poi chiedi all'API")

Model vs Entity:

- **Model** (Data Layer) — rappresentazione dei dati così come arrivano dall'API o dal DB (es. con `fromJson`)
- **Entity** (Domain Layer) — oggetto di business puro, senza dipendenze da framework o formati esterni

Single Source of Truth — i dati hanno un'unica fonte autoritativa. Tutti i componenti leggono da lì, mai da copie locali sparse.

MATERIA 7 — React Native

Fondamenti e Componenti Base

React Native — framework Meta per app native iOS e Android usando JavaScript/TypeScript e i paradigmi di React.

Componenti base ATTIVI (non deprecati):

- `<View>` — contenitore base strutturale (equivalente del `<div>` nel web)
- `<Text>` — obbligatorio per visualizzare qualsiasi testo a schermo
- `<FlatList>` — lista performante che renderizza solo gli elementi visibili (virtual scroll)
- `<Button>` — pulsante nativo di base
- `<ScrollView>` — area scrollabile per contenuti statici non troppo lunghi
- `<TextInput>` — campo di input testo
- `<TouchableOpacity>` — elemento cliccabile con effetto opacità
- `<Image>` — visualizzazione immagini
- `<Modal>` — finestra modale sovrapposta

Componenti DEPRECATI (non usare):

- `<Picker>` — menu a tendina (rimosso dal core, usare librerie esterne)
- `<ListView>` — sostituito da `<FlatList>` e `<SectionList>`

 **ESAME** — Deprecati: `Picker` e `ListView`. Attivi: `View`, `Text`, `FlatList`, `Button`.

Sintassi:

- Frammenti vuoti `<> ... </>` per raggruppare componenti senza wrapper aggiuntivi
 - Nidificazione libera: es. `<Text>` dentro `<ScrollView>`, `<View>` dentro `<FlatList>`
-

FlatList — Lista Performante

FlatList è il componente principale per liste in React Native. Renderizza solo gli elementi visibili a schermo, ottimizzando le performance con liste lunghe.

jsx

```
• const DATA = [  
•   { id: '1', title: 'Primo elemento' },  
•   { id: '2', title: 'Secondo elemento' },  
• ];  
•  
• const Item = ({ title }) => (  
•   <View style={styles.item}>  
•     <Text style={styles.title}>{title}</Text>  
•   </View>  
• );  
•  
• export default function App() {  
•   const renderItem = ({ item }) => <Item title={item.title} />;  
•  
•   return (  
•     <FlatList  
•       data={DATA}  
•       renderItem={renderItem}  
•       keyExtractor={item => item.id}  
•       // Props utili:  
•       // numColumns={2}      — griglia a 2 colonne  
•       // horizontal={true}    — scorrimento orizzontale  
•       // onEndReached={loadMore} — infinite scroll  
•       // refreshing={isLoading} — pull to refresh  
•       // ListHeaderComponent={<Header />}  
•       // ListEmptyComponent={<Empty />}  
•     />  
•   );  
• }  
}
```

Navigazione in React Native

React Navigation — libreria standard per la navigazione in React Native.

Tipi di navigatore:

- **Stack Navigator** — navigazione a stack (avanti/indietro come un browser)
- **Tab Navigator** — barra tab in basso (es. Home, Profilo, Impostazioni)
- **Drawer Navigator** — menu laterale scorrevole

jsx

- import { NavigationContainer } from '@react-navigation/native';

- import { createStackNavigator } from '@react-navigation/stack';
- import { createBottomTabNavigator } from '@react-navigation/bottom-tabs';
-
- const Stack = createStackNavigator();
- const Tab = createBottomTabNavigator();
-
- // Stack Navigator
- function AppStack() {
- return (
- <Stack.Navigator initialRouteName="Home">
- <Stack.Screen name="Home" component={HomeScreen} />
- <Stack.Screen name="Details" component={DetailsScreen} />
- </Stack.Navigator>
-);
- }
-
- // Navigare tra schermate
- function HomeScreen({ navigation }) {
- return (
- <Button
- title="Vai ai dettagli"
- onPress={() => navigation.navigate('Details', { itemId: 42 })}
- />
-);
- }
-
- // Ricevere parametri
- function DetailsScreen({ route }) {
- const { itemId } = route.params;
- return <Text>ID: {itemId}</Text>;
- }

Hook (use) — I Principali

Gli Hook permettono di usare lo stato e altre funzionalità React in componenti funzionali.

useState — gestisce lo stato locale del componente:

jsx

- const [count, setCount] = useState(0);
- const [user, setUser] = useState(null);
- const [items, setItems] = useState([]);
-
- // Aggiornamento
- setCount(count + 1);
- setUser({ name: 'Mario', age: 30 });

useEffect — effetti collaterali (chiamate API, subscriptions, timer):

jsx

- // Eseguito solo al mount (array vuoto)
- `useEffect(() => {`
- `fetchData();`
- `}, []);`
-
- // Eseguito ogni volta che 'userId' cambia
- `useEffect(() => {`
- `fetchUser(userId);`
- `}, [userId]);`
-
- // Con cleanup (es. per rimuovere listener)
- `useEffect(() => {`
- `const subscription = subscribe();`
- `return () => subscription.unsubscribe(); // cleanup`
- `}, []);`

useCallback — memorizza una funzione per evitare ricreazioni inutili:

jsx

- `const handlePress = useCallback(() => {`
- `doSomething(id);`
- `}, [id]);`

useMemo — memorizza il risultato di un calcolo costoso:

jsx

- `const sortedItems = useMemo(() => {`
- `return items.sort((a, b) => a.name.localeCompare(b.name));`
- `}, [items]);`

useRef — riferimento mutabile che non causa re-render:

jsx

- `const inputRef = useRef(null);`

`inputRef.current.focus(); // focus sull'input`

useContext — accede a un Context senza prop drilling:

jsx

`const theme = useContext(ThemeContext);`

Creazione Progetto con Expo (**--template**)

Expo — toolchain che semplifica la creazione e il deploy di app React Native.

Comando di creazione:

- `npx create-expo-app NomeApp`
`npx create-expo-app NomeApp --template`

Con **--template** si può scegliere tra template predefiniti:

- **Blank** — progetto vuoto minimale
- **Blank (TypeScript)** — progetto vuoto con TypeScript configurato
- **Navigation (TypeScript)** — con React Navigation già configurato
- **Default** — template standard con alcune funzionalità

Comandi principali:

- `npm install` # installa/aggiorna dipendenze
 - `npm start` # avvia il server di sviluppo (o expo start)
 - `npm run android` # avvia su emulatore Android
- `npm run ios` # avvia su simulatore iOS

Expo Go — app su smartphone reale. Scansiona il QR code generato da `npm start` → vede l'app in diretta con aggiornamenti in tempo reale (Hot Reload).

Snack — IDE React Native nel browser (snack.expo.dev), zero installazione, per testare codice e condividerlo al volo.

Expo Router — Navigazione File-Based

Expo Router — sistema di navigazione basato sulla struttura delle cartelle (come Next.js per il web). Ogni file nella cartella `app/` diventa automaticamente una rotta.

- `app/`
 - `_layout.tsx` → layout radice (NavigationContainer)
 - `index.tsx` → rotta '/' (home)
 - `about.tsx` → rotta '/about'
 - `profile/`
 - `index.tsx` → rotta '/profile'
 - `[id].tsx` → rotta '/profile/123' (parametro dinamico)
- tsx

- `// app/index.tsx` — schermata home
- `import { Link } from 'expo-router';`
-
- `export default function HomeScreen() {`
- `return (`

- `<View>`
- `<Text>Home</Text>`
- `<Link href="/about">Vai ad About</Link>`
- `<Link href="/profile/42">Vai al profilo</Link>`
- `</View>`
- `);`
- `}`
-
- `// app/profile/[id].tsx — rotta con parametro`
- `import { useLocalSearchParams } from 'expo-router';`
-
- `export default function ProfileScreen() {`
- `const { id } = useLocalSearchParams();`
- `return <Text>Profilo ID: {id}</Text>;`
- `}`
-
- `// Navigazione programmatica`
- `import { router } from 'expo-router';`
- `router.push('/about');`
- `router.replace('/login');` // sostituisce senza tornare indietro

`router.back();` // torna alla schermata precedente

Pezzo di Codice React Native — Lettura Critica

Per l'esame potresti dover leggere un pezzo di codice e identificare errori o cosa fa. Punti da controllare sempre:

Errori sintattici comuni:

- `<Text>` dentro `<View>` — corretto
- Testo diretto dentro `<View>` senza `<Text>` — **ERRORE** in React Native (nel web funziona, in RN no)
- `<ScrollView>` con tag non chiuso — errore
- Usare `<Picker>` o `<ListView>` — deprecati, errore concettuale

Struttura corretta di un componente:

jsx

- `import React, { useState, useEffect } from 'react';`
- `import { View, Text, FlatList, StyleSheet, TouchableOpacity } from 'react-native';`
-
- `export default function MyComponent() {`
- `const [data, setData] = useState([]);`
- `const [loading, setLoading] = useState(true);`
-
- `useEffect(() => {`

```

• fetch('https://api.example.com/data')
• .then(res => res.json())
• .then(json => {
•   setData(json);
•   setLoading(false);
• });
• }, []);
•
• if (loading) return <Text>Caricamento...</Text>;
•
• return (
•   <View style={styles.container}>
•     <FlatList
•       data={data}
•       keyExtractor={item => item.id.toString()}
•       renderItem={({ item }) => (
•         <TouchableOpacity onPress={() => console.log(item.id)}>
•           <Text>{item.name}</Text>
•         </TouchableOpacity>
•       )}
•     />
•   </View>
• );
• }
•
• const styles = StyleSheet.create({
•   container: { flex: 1, padding: 16 },
});

```

Cose da sapere per il codice:

- I componenti React Native usano `StyleSheet.create()` per gli stili (non CSS)
- Le dimensioni si esprimono in density-independent pixels (no `px`, solo numeri)
- `flex: 1` fa occupare tutto lo spazio disponibile
- Gli event handler si chiamano `onPress` (non `onClick` come nel web)
- `keyExtractor` in `FlatList` deve restituire una stringa univoca per ogni elemento

MATERIA 8 — Sviluppo Android Nativo (Kotlin & Jetpack Compose)

Approccio Classico (XML + Kotlin)

Gradle — sistema che gestisce automaticamente le dipendenze tramite il file `build.gradle.kts`. Scarica, integra e configura le librerie di terze parti (es. Retrofit, Glide).

Activity — modulo che gestisce una singola schermata. Avvio nuova Activity:
`startActivity(intent)`

Ciclo di vita dell'Activity: `onCreate` → `onStart` → `onResume` → `onPause` → `onStop` → `onDestroy`

Layout principali:

- `ConstraintLayout` — posizionamento flessibile con vincoli/ancoraggi — raccomandato
- `LinearLayout` — disposizione rigida orizzontale o verticale

Inflating — operazione con cui Android traduce i file XML in oggetti visuali in memoria. Nel modello classico: `setContentView(R.layout.activity_main)` dentro `onCreate`.

RecyclerView + Adapter — lista fluida per grandi quantità di dati. L'Adapter prende i dati grezzi → li mappa nel layout delle righe tramite ViewHolder → RecyclerView visualizza riciclando le righe non visibili per risparmiare memoria.

ViewPager + Fragment — interfacce a scorrimento laterale (es. guide introduttive, tab). I Fragment sono sotto-schermate con ciclo di vita autonomo. Nota: i Fragment creati automaticamente in Android Studio partono con un `FrameLayout` che va convertito in `ConstraintLayout`.

Librerie principali:

- **Retrofit** — chiamate HTTP verso REST API con conversione JSON automatica in oggetti Kotlin
- **Glide** — caricamento, caching e visualizzazione efficiente di immagini remote da URL. `implementation('com.github.bumptech.glide:glide:4.16.0')`

AndroidManifest.xml — file di configurazione globale. Per le chiamate di rete aggiungere obbligatoriamente:

xml

```
<uses-permission android:name="android.permission.INTERNET" />
```

Gestione risorse:

- Stringhe → `res/values/strings.xml` (per accessibilità e traduzione)
- Immagini vettoriali (SVG da Figma) → `res/drawable`
- Icone app → `res/mipmap` (SOLO icone, non immagini generiche)

Debug:

- **Logcat** — console di sistema con tutti i log. Si aggiungono log personalizzati nel codice
 - **Breakpoint** — bloccano l'esecuzione riga per riga per individuare errori logici
 - **Emulatore** — usare profili "Pixel" che includono i Google Play Services
-

Jetpack Compose (Approccio Moderno)

Jetpack Compose — framework dichiarativo moderno per UI Android. Tutto in Kotlin, senza file XML.

@Composable — funzione che genera una porzione di UI. Si configura passando parametri nelle parentesi tonde.

@Preview — mostra l'anteprima grafica nell'IDE senza avviare l'emulatore. Si possono mettere più preview con opzioni diverse.

Contenitori di layout:

- **Row { }** — dispone elementi in successione orizzontale
- **Column { }** — dispone elementi in successione verticale
- **Modifier** — oggetto per configurare stile, padding, dimensioni, click, forma del componente

kotlin

```

• @Composable
• fun UserCard(name: String, age: Int) {
•     Row(
•         modifier = Modifier
•             .fillMaxWidth()
•             .padding(16.dp)
•             .background(Color.White)
•     ) {
•         Column {
•             Text(text = name, fontWeight = FontWeight.Bold)
•             Text(text = "Età: $age", color = Color.Gray)
•         }
•     }
• }
•
• @Preview(showBackground = true)
• @Composable
• fun UserCardPreview() {
•     UserCard(name = "Mario", age = 30)
• }

```

Architettura MVVM:

- **Model** — dati puri e logica di business
 - **View** — funzioni composabile che mostrano i dati e catturano gli input
 - **ViewModel** — gestisce e trasforma i dati della UI, sopravvive ai cambi di configurazione (rotazione schermo)
-

MATERIA 9 — Sviluppo iOS Nativo (Swift & SwiftUI)

Swift — Basi del Linguaggio

Caratteristiche: Swift è fortemente tipizzato, moderno e orientato alla sicurezza (TypeSafe).

- `var` — variabile mutabile
- `let` — costante immutabile

Struct vs Classe:

Aspetto	Struct	Classe
Tipo	Value Type (copia indipendente)	Reference Type (stessa area di memoria)
Costruttore	Implicito (generato automaticamente)	Esplicito (bisogna scrivere <code>init</code>)
Ereditarietà	Non supportata	Supportata
Mutabilità	Immutabile di default	Mutabile

📌 **ESAME — Protocolli: CLASSI, STRUCT ed ENUM possono tutti conformarsi. Non solo classi e struct!**

Tipi di dati:

- **Enum** — tipo con set chiuso di casi predefiniti
- **Tuple** — raggruppa più valori temporaneamente, senza metodi né costruttori

- **Array** — lista ordinata tramite indici | **Dictionary** — coppie chiave-valore | **Set** — valori unici non ordinati

Proprietà:

- **Stored Property** — memorizza fisicamente un valore
- **Computed Property** — calcola il valore ogni volta che viene chiamata, non occupa spazio in memoria

Optional (?) e Unwrapping:

- Una variabile optional può contenere un valore oppure essere `nil`
- `if let` — spacchetta nel blocco if, valore disponibile solo dentro il blocco
- `guard let` — spacchetta all'inizio della funzione; se nil esce con `return`. Il valore è disponibile per tutto il resto della funzione

Altri concetti chiave:

- **Codable** — protocollo per serializzazione/deserializzazione JSON automatica
 - `async / await` — gestione asincrona moderna
 - **Extension** — aggiunge funzionalità a classi/struct/tipi esistenti senza modificare il codice originale
 - **Singleton** — `static let shared = NomeClasse() + private init()`
-

SwiftUI — Interfacce Dichiarative

- **View** — ogni schermata è una struct conforme al protocollo View con proprietà computata `body`
- **@State** — variabile monitorata: se cambia, SwiftUI ridisegna automaticamente la vista

Contenitori di layout:

- **VStack** — Vertical Stack: elementi uno sotto l'altro
- **HStack** — Horizontal Stack: elementi uno accanto all'altro
- **ZStack** — Depth Stack: elementi sovrapposti (primo dichiarato = sfondo)

Componenti:

- **List + ForEach** — lista nativa scrollabile con separatori automatici. ForEach itera su un array
- **AsyncImage** — carica immagini remote da URL in background senza bloccare la UI
- **UserDefaults** — storage chiave-valore per piccole preferenze utente (es. tema scuro)